

HACKATHON PROJECT BRIEF

QueueLess

An AI agent that manages the waiting room, not just the queue

A simple guide for the team and a build spec for our AI coding agent: the problem, the features, the scenarios, the architecture and the order to build things in.

The idea in one line

"QueueLess doesn't just tell people how long they will wait. It actively helps service providers reduce that waiting time when conditions change."

How to read this document

Reader	Read these sections
Teammates (understand and agree on the idea)	1 to 6: idea, problem, features, the full demo story, and the worked example
AI coding agent / IDE (build the project)	Everything, especially 7 to 12: architecture, data model, agent tools, rules, build order and acceptance checklist
Anyone preparing the presentation	13 and 14: the demo script, the pitch and judge Q&A

Contents

1	The idea in simple words	8	Architecture: what is AI and what is not
2	The problem and who it hurts	9	Data model, agent tools and screens
3	The core loop of the agent	10	Rules the system must always follow
4	The demo world: CityCare clinic	11	Build plan for 10 to 15 hours
5	The features (with scenarios)	12	Acceptance checklist
6	The full 10:00 to 10:40 story	13	Demo script and pitch
7	Worked example: approving a reassignment	14	Honest positioning, risks and glossary

1. The idea in simple words

Most queue apps are **token systems**. They answer one question: **“Who is next?”**

QueueLess answers a better question: **“Given what is happening right now, what should we change to reduce waiting time, keep things fair, and keep the queue moving?”**

Think of a good floor manager in a busy clinic. They do not just read out numbers. They notice that one desk is stuck, see that another desk is free, move a patient across, and explain why. QueueLess is that floor manager, as software. It watches the queue, predicts what a disruption will cause, suggests a fix, asks a human to approve, makes the change, and records everything.

A normal token system	QueueLess
Shows who is next	Shows who is next and notices when the queue is going wrong
ETA = people ahead × average time	ETA built from counter status, service type and live changes, shown as a range
Operator has to figure out the fix	Agent recommends a fix with the expected before/after wait
Changes happen silently	Every change needs approval where required, and is explained and logged

What we are building

QueueLess: an AI-powered adaptive queue operations agent. It treats a queue as a system that keeps changing, not a fixed line of tokens. It watches: people waiting, service speed, counter availability, appointment delays, no-shows, and changing wait predictions. Then it recommends or performs controlled actions to keep the queue efficient and fair.

Be honest about what is new

Live tokens, ETA, notifications and no-show detection already exist in many products. **We must not claim to be the first AI queue system.** What we can present is the **combination**: a live disruption, constraint-aware action selection, a quantified before/after prediction, a human-approved fix, and an auditable explanation of why the system acted. We also cannot claim that no competitor offers this exact combination.

2. The problem and who it hurts

A queue never runs exactly to plan. Someone takes longer, someone does not turn up, a crowd arrives. Today the waiting-room staff handle this by guesswork and the patient is left in the dark.

Who	What goes wrong today	What QueueLess gives them
Patient / customer	Told “15 minutes”, then it quietly becomes 30. Does not know why. Cannot plan.	An honest wait range that updates, a short reason, and a message when it materially changes.
Receptionist / operator	Sees a long queue but does not know the best fix. One counter is overloaded while another sits idle.	A clear recommendation, the expected saving, and an Approve / Reject button.
Clinic owner / manager	No record of why decisions were made. Hard to improve the process.	An audit trail of each event, recommendation, decision and result. A what-if tool for planning.

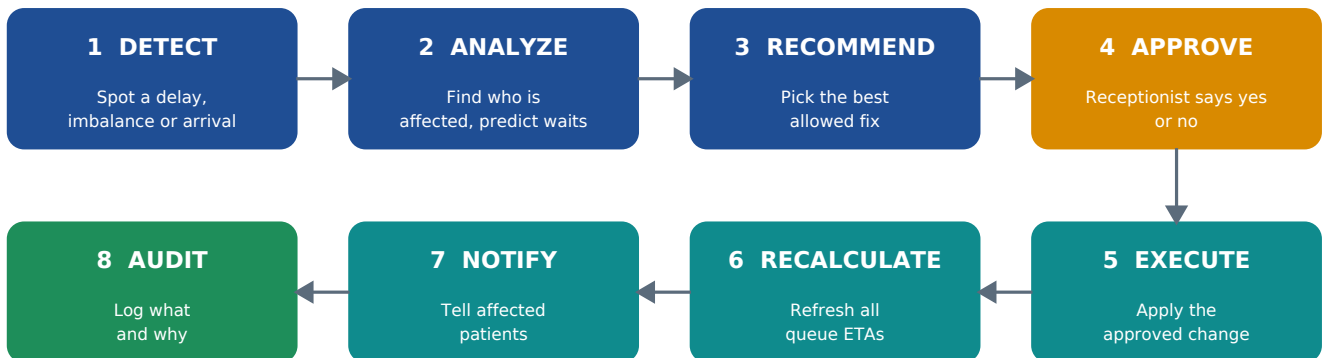
Typical situations (use cases)

- **A patient takes longer than expected**, so everyone behind them waits longer.
- **One counter is overloaded while another is free** and qualified to help.

- **An appointment patient arrives late** and the queue order must stay fair.
- **A called patient does not show up**, wasting a service slot.
- **Many walk-ins arrive suddenly** and staff wonder if opening another counter would actually help.

3. The core loop of the agent

Every feature in QueueLess is the same loop applied to a different situation. If the team remembers one picture, it should be this one.



Rejection is part of the loop. If the receptionist rejects a recommendation, nothing is forced. The queue stays as it is, the rejection is logged, and the agent may suggest another option. This shows *controlled autonomy*: the AI helps, humans stay in charge.

4. The demo world: CityCare Diagnostic Centre

We prove the whole idea with **one small clinic**, because it naturally has several service types, different service durations, no-shows and real disruptions. All data is **synthetic**. We are not using any real clinic data.

Setup

- 2 service counters, 12 registered patients, a mix of walk-ins and appointments.
- A receptionist dashboard, a patient-facing live queue, and the queue operations agent.
- A way to **inject events live** (change a service time, add walk-ins, mark a no-show) so judges can try it.

Counters

Counter	Recent average service time	Status at 10:00	Can serve (suggested demo config)
Counter A	6 min	Available (busy with Q101)	Blood test, Consultation
Counter B	8 min	Available	Blood test, ECG

Service capabilities must be **data, not hard-coded**, so the validator can reject an impossible move (for example moving a consultation to Counter B in the demo). The original idea notes describe Counter A both as doing blood tests and as “consultations only”; the configuration above resolves that so every scenario works.

Starting queue (12 tokens)

Token	Service	Status	Token	Service	Status
Q101	Blood test	Being served at Counter A	Q107	ECG	Waiting
Q102	Consultation	Waiting	Q108	Consultation	Waiting (appointment)
Q103	Blood test	Waiting	Q109	Blood test	Waiting
Q104	ECG	Waiting	Q110	ECG	Waiting
Q105	Consultation	Waiting	Q111	Consultation	Waiting
Q106	Blood test	Waiting	Q112	Blood test	Waiting

Suggested default service durations for the seed data (our assumption, adjustable): Blood test 6 min, ECG 8 min, Consultation 10 min. Counter averages above can scale these. Add some natural variation so ETA ranges look real. Q108 is the late-arriving appointment patient in Scenario C.

5. The features

There are five features. **Four are headline features** for the prototype. The fifth is a supporting feature to build only if time remains. The most important rule: the **maths and rules are plain code**. The AI explains, chooses which tools to call, and coordinates. It never calculates times itself.

#	Feature	Role in the pitch	Priority
1	Live ETA with uncertainty	Foundation. Every other feature needs it.	Must build
2	Bottleneck Rescue Agent	The hero. Centre of the presentation.	Must build
3	Fairness-aware rules	The engineering differentiator.	Must build (minimal)
4	What-if simulator	The moment judges interact.	Must build
5	No-show recovery	Visual and nice, but supporting.	If time permits

FEATURE 1

Live ETA that understands uncertainty

MUST BUILD

In simple words

A basic app says: $ETA = \text{people ahead} \times \text{average time}$. That breaks as soon as counters serve different services or times change. QueueLess works out each patient's wait from what is really happening, and shows a **range** (for example “about 10 to 14 minutes”) instead of one fake-precise number.

It uses

- Current counter status and when each counter will be free
- Service-specific duration history
- How many people are waiting for each service
- Active appointments
- Recent changes in service times
- Counter availability

Scenario A: a patient takes longer than expected

At 10:00 Q101 is expected to finish at 10:06. At 10:04 the receptionist updates the service: Counter A's expected remaining time goes up by **12 minutes**. QueueLess recalculates (illustrative): Q103's ETA moves from **10:12 to 10:24**. Affected tokens: **Q103, Q106, Q109, Q112**.

What the system does

1. Finds which tokens are affected by the delay.
2. Recalculates their expected waits.
3. Updates the live display.
4. Notifies only patients whose ETA changed materially (a configurable threshold, for example 5 minutes or more).
5. Explains the disruption to the receptionist in one or two plain sentences.

Suggested ETA method (deterministic)

```
for each counter: free_at = now + expected_remaining_time(current_token)
for each waiting token, in queue order:
    pick the eligible counter with the earliest free_at
    token.eta_expected = that counter's free_at
    that counter's free_at += expected_duration(token.service)
range: eta_low / eta_high = eta_expected +/- k * spread
      (spread from the std deviation of recent durations for the services ahead)
```

Algorithm does	AI does
ETA calculation, uncertainty ranges, finding affected tokens	Writes the plain-language explanation and the patient message wording

Done when

Changing one service duration live updates ETAs and ranges on both the dashboard and the patient screen within a second or two, and the affected tokens are visibly highlighted.

FEATURE 2

Bottleneck Rescue Agent

MUST BUILD · HERO

In simple words

Traditional systems tell you the queue is long. QueueLess notices that an **operational problem is developing** and recommends a fix. It acts like a floor manager who sees one counter swamped while another is free.

Scenario B: Counter A is overloaded (10:10 AM)

- Counter A has a growing queue.
- Counter B just finished its patient and is free.
- Counter B is qualified to handle blood tests.
- Q103 and Q106 are waiting for blood tests.
- Q103's projected wait is now 22 minutes.

Agent recommendation (illustrative)

Reassign Q103 to Counter B, provided eligibility is confirmed.

Projected wait for Q103: **22 min before, 8 min after.**

[Approve reassignment] [Reject]

Numbers are illustrative. The real app must compute them from live state.

What the agent does

1. **Observe** live state (queue, counters, ETAs).
2. **Detect** a bottleneck: projected wait over a threshold, or one counter idle while another is overloaded.
3. **Generate candidate actions:** move a token, open another counter, or just notify patients of the delay.

4. **Validate** each candidate with the policy validator (service eligibility, appointments, priorities).
5. **Simulate** each valid candidate and compute the before/after wait.
6. **Recommend** the best one with a plain explanation, and **ask for approval**.
7. On approval: **execute**, recalculate ETAs, notify affected patients, write the audit log.

Algorithm does	AI does
Threshold and workload analysis, candidate generation, validation, simulation, predicted saving	Decides which tools to call, explains the recommendation using the computed numbers, handles the conversation. It must not invent the time saving.

Done when

A recommendation card appears with before/after numbers, Approve and Reject both work, approval changes the live queue and ETAs, rejection leaves the queue unchanged and is logged, and every step appears in the audit trail.

FEATURE 3

Fairness-aware queue rules

MUST BUILD · MINIMAL

In simple words

A naive optimizer always sends a patient to the fastest counter. That can break service eligibility, appointment rules or priority policy. QueueLess optimizes **inside explicit rules**. It is not “AI decides who goes first”. It is a rule-based or constraint-based scheduler that the AI can explain.

The rules (all configurable)

- **Service compatibility:** only counters that can do the service.
- **Appointment windows:** appointments are honoured within a grace period.
- **First come, first served** within the same priority class.
- **Staff-defined priority** only. The system never decides medical urgency.
- **Maximum extra delay** allowed for patients already waiting.

Scenario C: an appointment patient arrives late (10:15 AM)

Q108 arrives for a scheduled consultation while Q105 is already waiting. The agent does **not** push Q108 to the front. It checks: (1) Is the appointment within its grace period? (2) Is there reserved capacity? (3) Would inserting Q108 delay others too much? (4) Is the sequence consistent with clinic policy?

Proposed consultation order	Why
1. Q105, existing consultation	Already waiting, order preserved
2. Q108, appointment within grace period	Qualifies for appointment grace policy
3. Q111, next consultation	Continues normal order

Example agent explanation: “Q108 qualifies for the appointment grace policy. Placing Q108 after Q105 preserves the current service order while accommodating the appointment.”

Also show a rejection

In the demo, try moving a consultation token to Counter B (which cannot serve consultations). The validator must refuse and say why in plain words. This proves the system is safe, not just clever.

Done when

An invalid reassignment is rejected with a clear reason, a late appointment is inserted according to the grace rule, and normal first-come-first-served order is preserved otherwise.

FEATURE 4**“What if?” operations simulator****MUST BUILD****In simple words**

The receptionist can ask: “*What happens if we open a third counter for 20 minutes?*” QueueLess runs a copy of the queue forward with the proposed change and compares it with the current setup. It is a **simulation of a possible future**, not a claim that we reduced waiting in a real clinic. It is also the best moment to let judges play.

Scenario E: five more walk-ins arrive (10:30 AM)

	Current setup	Proposed setup
Counters	2	3
Projected average wait (illustrative)	34 min	21 min
Difference		about 13 min less per patient on average

These numbers are examples. The implementation must **calculate its own results**.

The simulator should show

- Current queue with the current counters
- Proposed queue with the changed setup
- Expected wait distribution (or at least average and range)
- How many patients are affected
- Whether the extra counter is actually useful (it may not be!)

What judges can change

- Number of extra patients
- Number of counters or counter capacity
- A service delay
- A no-show or a sudden arrival spike

Build tip

Reuse the same ETA and scheduling code from Feature 1 on a **copy** of the queue state. The simulator must never touch the live queue. Optionally run the copy several times with random service durations to get a range.

Done when

Changing the inputs updates the comparison immediately, the live queue is untouched, and the result is clearly labelled as a simulation.

FEATURE 5

No-show recovery agent

SUPPORTING · IF
TIME

In simple words

A basic system just marks an absent patient as absent. QueueLess thinks about the **consequence**: a service slot is wasted, so it frees the slot and offers it to the next eligible patient.

Scenario D: Q104 does not arrive (10:20 AM)

1. Q104 is called to Counter B and marked **CALLED**.
2. A configurable grace timer starts.
3. A reminder is sent if the patient opted in.
4. If the grace period ends, the token becomes **NO_SHOW**.
5. The queue is recalculated.
6. The freed slot is offered to the next eligible token (here Q106, estimated delay avoided about 8 minutes, illustrative).
7. Q104's record is kept, and staff can **restore** the token if the patient returns.

Token states: WAITING → CALLED → IN_SERVICE → COMPLETED; CALLED → NO_SHOW → (staff restore) → WAITING.

Do not spend the hackathon building notifications while the central intelligence (Features 1 to 4) is unfinished. Real WhatsApp or SMS delivery is optional; simulated notifications in the UI are enough for the demo.

6. The full story: CityCare 10:00 to 10:40

The six features are not separate demos. They are one story on the same clinic, one disruption after another.

Time	What happens	What QueueLess does	Feature
10:00	Normal queue. Q101 is being served. 12 patients registered.	Estimates waits from service times and counter state.	ETA engine
10:04	Q101 needs 12 more minutes.	Finds affected tokens, recalculates ETAs, updates patients.	Dynamic ETA
10:10	Counter A overloaded, Counter B free.	Recommends moving Q103, computes impact, asks to approve.	Bottleneck rescue
10:15	Appointment patient Q108 checks in.	Checks eligibility and grace rule, inserts without breaking the order.	Fairness rules
10:20	Q104 does not respond when called.	Grace timer ends, releases slot, finds next eligible patient.	No-show recovery
10:30	Five walk-ins arrive.	Runs what-if: two counters vs a proposed third.	What-if simulator
10:40	End of the story.	Shows event timeline, accepted/rejected recommendations, projected vs actual waits, reasons.	Audit

7. Worked example: approving the Q103 reassignment

This is exactly what the prototype should do when the receptionist clicks **Approve** in Scenario B. Values are an illustration. The real system recalculates them.

What changes

	Before	After
Q103 location	Counter A queue (blood test)	Counter B, next to serve (reassigned from A)
Q103 ETA	22 min	8 min (14 min earlier)
Q106 ETA	28 min	20 min
Q109 ETA	34 min	26 min
Q112 ETA	40 min	32 min

Audit trail the system must write

Time	Step	Detail
10:10:00	Bottleneck detected	Counter A's projected waiting time exceeded the configured threshold.
10:10:02	Capacity checked	Counter B is available and qualified for blood tests.
10:10:04	Alternatives evaluated	Scheduler compared keeping Q103 on A vs reassigning to B.
10:10:05	Recommendation generated	Reassign Q103. Estimated improvement: 14 minutes.
10:10:12	Human approval	Receptionist approved.
10:10:13	Queue updated	Q103 moved to Counter B. State saved.
10:10:14	Notifications triggered	Q103 got new counter and ETA. Other affected patients got revised estimates.
10:10:15	Action completed	Previous assignment, new assignment, approval and projected impact recorded.

What the patient sees

Your queue has been updated · CityCare Diagnostic Centre

Token **Q103** · Estimated wait: **8 minutes** · Proceed to **Counter B** when called

"We have moved you to an available service counter to reduce your waiting time."

If the receptionist rejects

- Q103 stays in its original queue and its ETA keeps updating live.
- The rejection is recorded in the audit log.
- The agent may suggest another action, such as opening another counter or notifying patients of the delay.

8. Architecture: what is AI and what is not

Data flows from check-in and the receptionist, through a queue API into a state database and a real-time event stream. Deterministic engines (ETA, bottleneck detector, validator) feed the agent. The agent proposes actions; the validator checks them; a human approves where needed; the action executes; screens and the audit log update.

Layer	Component	Job
Input	Patient check-in / Receptionist UI	Create tokens, update service status, inject demo events
Backend	Queue API + Queue State Database	Single source of truth for tokens, counters, policies
Backend	Real-time event stream (WebSocket / Socket.IO)	Push every change to dashboards and patient screens
Engine	ETA Prediction Engine	Deterministic ETA and ranges
Engine	Bottleneck Detector	Thresholds and workload analysis
Brain	Queue Operations Agent (LLM with tools)	Chooses tools, explains, coordinates
Safety	Policy and Constraint Validator	Blocks any action that breaks a rule
Gate	Human approval check	Operational changes need approval; safe permitted actions can run automatically
Output	Live Patient Display, Operations Dashboard, Decision & Audit Log	Show state, recommendations and reasons

What uses AI and what does not

Component	Technology
ETA calculation	Deterministic algorithm
Service-time learning	Rolling or weighted average
Bottleneck detection	Thresholds + workload analysis
Queue optimization	Rule-based or constraint-based scheduler
Safety and service eligibility	Deterministic rules
Natural-language receptionist commands	LLM
Explanation of a recommendation	LLM , grounded in the actual calculated numbers
Customer notifications	Messaging API (or simulated in the UI)
Live queue updates	Backend + WebSocket

If a judge asks “Why do you need AI? Why not normal code?”

“The scheduling and prediction logic is deterministic. The agent interprets operational context, selects which diagnostic or planning tools to invoke, explains trade-offs, and coordinates approved actions. We use AI where language and flexible reasoning matter, not where exact calculations are required.”

9. Data model, agent tools and screens

Everything below is a **suggested starting point** for the AI coding agent. Rename or adjust freely as long as the rules in section 10 stay true.

Suggested data model

Entity	Key fields
ServiceType	id, name (Blood test, ECG, Consultation), avg_duration, spread, recent_durations[]
Counter	id, name, status (AVAILABLE, BUSY, CLOSED), supported_services[], current_token, expected_free_at
Token	id (Q101), service, kind (WALK_IN or APPOINTMENT), appointment_time, priority_class (staff-set), status, assigned_counter, created_at, eta_low, eta_expected, eta_high
Policy (config)	bottleneck_threshold_min, notify_change_threshold_min, appointment_grace_min, max_delay_existing_min, no_show_grace_min, actions_requiring_approval[]
Recommendation	id, type (REASSIGN, OPEN_COUNTER, NOTIFY), token, from, to, wait_before, wait_after, reason, status (PENDING, APPROVED, REJECTED)
AuditEntry	timestamp, event_type, actor (SYSTEM, AGENT, RECEPTIONIST), input, prediction, recommendation, decision, result

Suggested agent tools (functions the LLM may call)

Tool	What it does (all logic is plain code)
get_queue_state()	Returns tokens, counters, statuses
get_eta_forecast(token?)	Returns ETA and range from the ETA engine
find_bottlenecks()	Returns overloaded and idle counters with numbers
validate_action(action)	Returns allowed or blocked, with the reason
simulate_action(action scenario)	Returns before/after waits using a copy of the state
propose_action(action)	Creates a PENDING recommendation shown to the receptionist
execute_approved_action(id)	Applies only an APPROVED recommendation, recalculates, notifies, logs
get_audit_log()	Returns recent decisions for explanation

Suggested real-time events

queue:updated · eta:updated · recommendation:created · recommendation:decided · token:called · token:no_show · audit:entry

Screens

Screen	Contents
Receptionist dashboard	Queue board per counter, ETAs with ranges, agent panel (recommendations with Approve/Reject), what-if panel, demo controls (inject delay, add walk-ins, mark no-show, late appointment)
Patient live view	Enter a token, see counter, ETA range, and a short reason when it changes
Audit / timeline view	Event log: input, prediction, recommendation, approval, action, updated queue; projected vs actual waits

10. Rules the system must always follow

These are non-negotiable. They are what make the project credible instead of a chatbot with a queue.

#	Rule	Meaning
1	The LLM never does the maths.	ETAs, ranges, savings and simulations come from code. The LLM only explains them.
2	Predicted savings are never invented.	Every before/after number shown comes from a real calculation on the current state.
3	Humans approve operational changes.	Reassigning patients or changing counters needs receptionist approval unless the policy marks the action as safe.
4	Rules beat speed.	The validator can veto any recommendation. Service eligibility, appointment rules and staff priorities are never bypassed.
5	No medical prioritization by AI.	Urgency is decided by qualified clinic staff under clinic-defined policy. The system only follows staff-set priority.
6	Rejection must work.	A rejected recommendation leaves the queue unchanged, is logged, and the app keeps functioning.
7	Everything is logged.	Input, prediction, recommendation, decision, action and result are all written to the audit log.
8	Simulations are labelled.	What-if results and demo numbers say “simulation / illustrative”. Never present them as measured real-world results.
9	Thresholds are configuration.	Bottleneck limits, notification thresholds, grace periods and delay limits live in a policy config, not in code.
10	The core demo does not depend on external messaging.	Notifications can be simulated in the UI. WhatsApp or SMS is optional.

11. Build plan for 10 to 15 hours

Use **synthetic clinic data** and a **real external LLM API** with tool calling. Build in this order so something demo-able exists early.

Feature	Priority	Difficulty
Token creation and service types	Required	Easy
Live queue via Socket.IO	Required	Medium
Dynamic ETA	Required	Medium
Bottleneck recommendation	Required	Medium
Approval and reassignment	Required	Medium
What-if simulation	Required	Medium
AI explanation / tool calling	Required	Medium
Fairness rules	Minimal version	Medium
No-show recovery	Supporting, if time permits	Medium
WhatsApp notifications	Optional	Medium to high
Advanced prediction model	Exclude	High
Real clinic integration	Exclude	High

Suggested build order

Phase	Build	Result you can demo
1	Seed data, token and counter state, dashboard board, Socket.IO live updates, demo controls	A live queue that reacts to button presses
2	ETA engine with ranges, “inject delay” control, affected-token highlight	Scenario A works
3	Bottleneck detector, minimal validator, recommendation card, approve/reject, execute, audit log	Scenario B works end to end
4	What-if simulator on a copy of state, comparison panel	Scenario E works
5	LLM agent with tools: explanations and natural-language commands, grounded in tool results	The agent explains and acts through tools
6	Fuller fairness rules (late appointment, grace period). Then no-show recovery. Then optional notifications	Scenarios C and D

Suggested stack (change freely): Node.js + Express + Socket.IO for the backend, SQLite or in-memory state, a simple React or plain-JS frontend, and an LLM API with function calling.

12. Acceptance checklist

The prototype is ready when all of these are true:

- ☐ Twelve tokens, two counters, and live ETAs with ranges are visible.
- ☐ Changing one service duration updates ETAs and highlights the affected tokens.
- ☐ The agent detects a bottleneck and shows a recommendation with before/after numbers computed by code.
- ☐ Approve changes the queue, ETAs, patient message and audit log. Reject changes nothing except a logged rejection.
- ☐ An invalid reassignment (wrong service) is blocked by the validator with a plain reason.
- ☐ A late appointment is placed according to the grace rule without breaking normal order.
- ☐ The what-if simulator compares two vs three counters, responds to changed inputs, and never touches the live queue.
- ☐ The audit view shows input, prediction, recommendation, approval, action and updated queue for each event.
- ☐ All simulated or illustrative numbers are labelled as such.

13. Demo script and pitch

Do not start with the dashboard. Start with the problem.

90-second live demo

Time	Step	What to show
0 to 15 s	Show the queue	Twelve tokens, two counters, live ETA
15 to 30 s	Inject a disruption	Change one service from 6 to 18 minutes. ETAs change.
30 to 50 s	Ask the agent to fix it	Bottleneck recommendation with the simulated impact of moving Q103
50 to 65 s	Challenge its decision	Add an appointment or a service-incompatible counter. Validator rejects an invalid move.
65 to 80 s	Run a what-if	Add five walk-ins. Compare two vs three counters.
80 to 90 s	Show proof	Event log: input, prediction, recommendation, approval, action, updated queue

The most impressive moment is not the AI saying something clever. It is when a judge changes a live condition and QueueLess responds with a **measurable, explainable change**.

The pitch (say it aloud)

QueueLess: from queue tracking to queue intelligence

Imagine walking into a clinic and getting token Q103. You are told you will wait 15 minutes. But the patient ahead takes longer, and suddenly it is 30. The receptionist knows the queue is delayed but not the best way to fix it.

Existing queue systems mostly help businesses digitize the waiting line. We want to go one step further.

QueueLess is an AI-powered queue operations agent that continuously monitors service conditions, predicts disruptions, and helps staff take corrective actions. When a counter is overloaded it checks whether a patient can move to another eligible counter. When an appointment arrives it respects the clinic's scheduling rules. When someone misses their turn it releases the slot and recalculates. And when demand spikes, our what-if simulator shows whether opening another counter will really help.

Every recommendation comes with its reasoning, its projected impact, and a human approval step.

Our goal is not to show people where they stand in a queue. It is to help the queue recover intelligently when reality does not go to plan.

14. Honest positioning, risks and glossary

What makes this project strong

- Judges understand the problem in seconds, and the workflow is easy to see on screen.
- AI has a clearly defined job, and the backend contains real algorithms and state changes.
- A small team can finish a coherent MVP in the time available.
- It lets us talk about real-time systems, concurrency, prediction, optimization and agent orchestration afterwards.

What could make us lose

Risk	How we avoid it
The agent is just a chatbot	Give it tools, a real state, and real actions with approval
Projected time savings are made up	Compute every number in code from the live state
The algorithm is only FIFO with an AI label	Show constraint checks, candidate comparison and simulation
The demo cannot handle a rejected recommendation	Build and rehearse the reject path
We claim novelty that existing products already cover	Pitch the combination, not “first AI queue”

Strongest advice

Make the **Bottleneck Rescue Agent** the hero, the **What-if Simulator** the judge-interaction moment, and **fairness-aware scheduling** the engineering differentiator.

Glossary

Term	Meaning
Token	A queue number given to a patient, like Q103
ETA	Estimated time until the patient is served
Bottleneck	A point where work piles up while capacity elsewhere is unused
No-show	A patient who was called but did not arrive within the grace period
Grace period	Extra time allowed before a late arrival or non-response is treated as a miss
Constraint	A rule the schedule must obey, such as “only counters that offer this service”
Validator	Code that checks every proposed action against the rules before it can run
Audit log	A permanent record of what happened, what was decided, and why
What-if simulation	Running a copy of the queue forward under a changed condition to compare outcomes